SOP: Adding a New Document Category

Process document C(i). Applies to the SunGrid AI Copilot repository. Commands run from the repository root with the virtualenv active and `ANTHROPIC_API_KEY` set in `.env`. Roles are assumed, since the brief names no team: **Content Owner** (SunGrid business person who owns the documents), **AI/DS Associate** (does the technical work), **Lead** (reviews and approves the change), **Support Lead** (SunGrid support manager who signs off that answers are right for members).

When to use this. A new group of documents does not fit any of the five categories (`program_policies`, `incentive_rebate`, `billing_account`, `technical_installation`, `company_updates`). If the documents fit an existing category, add them to it using steps 4 to 7 only. If the new category also needs a service call (like eligibility), stop: that is a design change, not this SOP. Raise it with the Lead.

Steps

#	Step	Owner	Done when
1	**Intake.** Content Owner sends the documents as Markdown with one ## heading per topic, plus a one-line definition of the category and 5 example member questions that belong in it and 3 that look similar but do not.	Content Owner	Ticket exists with all three attached. Documents are final: nothing marked draft, no placeholder text.
2	**Fit check.** Read the documents against the existing categories. Does any document straddle two categories (as 06 does)? Does any contradict an existing document (as 01 and 08 do on voting weight)? Does anything refer to a section or appendix that does not exist?	AI/DS Associate	A short note lists every straddling document, conflict and dangling reference. Conflicts go back to the Content Owner to resolve **before** ingestion. Unfinished content gets a `section_overrides` entry in the registry (status `unreleased`) so it is never quoted as policy.
3	**Name and define the category.** Choose a snake_case id. Write the definition and examples as in `copilot/prompts/classifier_v1.md`. Say what separates it from its nearest existing category.	Associate drafts, Lead approves	The definition names the nearest neighbouring category and the rule that separates the two.
4	**Branch and copy files.** Create a branch. Add the documents to `docs/` using the existing numbering, e.g. `15_name.md`. Do not edit the documents' wording: if wording is wrong, send it back to the Content Owner.	Associate	Branch exists, files added unchanged.
5	**Register.** In `data/doc_registry.yaml`, add the category id to the top-level `categories` list. Add one entry per new document with `categories` (primary first, then any secondary) and a one-line `rationale`.	Associate	`python -c "from copilot.registry import load_registry; load_registry()"` runs without error. It fails if a file is missing from the registry, a registry entry has no file, or a label is not in the list.
6	**Update the code that lists categories.** Add the new id to the two `Literal` lists in `copilot/classifier.py` (`Category`, `Topic`). Add its definition, examples and its boundary with the nearest category to `copilot/prompts/classifier_v1.md`. Follow the change-control SOP (C2) for the prompt change, as this is a prompt change.	Associate	The diff touches only those places. `pytest` passes (44 tests at the time of writing, no API calls).
7	**Ingest.** Run `python -m copilot.ingest`. Check the printed chunk count equals the old count plus the number of ## sections in the new documents. Spot-check three chunks in `qdrant_data` for the right title, heading and categories.	Associate	Counts match. Every new chunk carries the new category in its payload.
8	**Extend the evaluation set** (`eval/questions.yaml`). Add at least 3 questions per new category, covering: a plain lookup, a question whose answer needs two facts, and a near-neighbour question that belongs in a different category. Add one more per straddling document. Give each `gold_chunks` and `answer_facts`. Add 1 unanswerable question about the new area. Write them from the documents **before** looking at any model output. A different person from whoever wrote the prompt text should write at least half.	Associate and Support Lead	`pytest tests/test_eval_set.py` passes (it checks that every gold chunk exists and the brief's coverage rules still hold).
9	**Run the evals** and read every failure, not just the totals: `python -m eval.run_eval retrieval`, then `classifier`, `eligibility`, `e2e` (live, 3 runs each; the whole set costs roughly $0.12 at current prices). Save the reports under `eval/reports/`.	Associate	Pass thresholds below are met, and every failure is explained in the ticket.
10	**Review.** Lead reviews the diff, the reports and the failure notes. Support Lead reads the answers to the new questions (the e2e report lists them) and confirms each is correct and safe to give a member.	Lead, Support Lead	Both approve in the ticket.
11	**Release.** Merge. Run ingestion against the production Qdrant. Smoke test: ask each new example question through the live Copilot and compare with the eval run.	Associate runs, Lead merges	The smoke test answers match. Release is recorded in the changelog.
12	**Watch.** For the first 2 weeks, review every trace in the new category weekly plus the usual 5% sample. Add any real question that went wrong to the eval set.	Associate	Review note filed at 1 and 2 weeks.

Validation checks and pass thresholds

These are starting values chosen by the Lead, not measured, and are reset when the eval set grows. - **Classifier:** every new example question labelled correctly in all 3 runs. No old question changes its label. Existing categories keep 60/60 on

category and intent; any new miss is investigated even if it looks harmless. - **Retrieval:** every new gold chunk in the top 5 for every new question. Existing recall@5 stays at its previous value (1.00 on the current set). - **End to end:** new questions pass in at least 2 of 3 runs, and the existing pass rate does not drop (66 of 69 at the time of writing, with q03 a known failure). Any previously passing question that fails is a block until explained. - **Safety:** the eligibility gate stays at 12 of 12 and `pytest` passes. A new category must never change the eligibility path. - **Cost and speed:** cost per question and median latency stay within 20% of the last report.

Sign-off record

Every ticket records: documents added (name and version), the registry diff, the prompt diff, the report files, any accepted failure with its reason, and the names and dates of Lead and Support Lead approval. No approval, no release.

If something goes wrong

Revert the merge, restore the previous registry and prompt from Git, re-run ingestion, and re-run `python -m eval.run_eval retrieval` to confirm the index is back. The vector store is derived entirely from the repository, so nothing is lost by rebuilding it.